

Question 1

1. Modern programming languages do not include `goto` because it violates structured programming principles. When using `goto`, there are many drawbacks, and one of the most prevalent is that it produces spaghetti code — essentially meaning the code is unpredictable and hard to follow since jumps are happening all over the place in different parts of the code. Both articles agree that the use of the `goto` statement is justified in certain cases, and the examples seen in the article are those of error handling and cleanup. The article **Linux Device Drivers** suggests that the `goto` statement is one of the more elegant ways to code exceptions into a C function since the error handling code does not clutter up the main code path. In the article **Linux: Using goto in Kernel Code**, Linus Torvalds argues that using `goto` is just much clearer and more readable since it essentially does what the algorithm states that it does without the need for nested conditionals.

2. Some of the key issues of the counter-controlled loop are where and how to declare the variable. Should the counter be declared inside the loop construct like in C++, or should it be declared outside? Also, the scope of the counter variable should be visible outside of the loop when the loop terminates or not. There is also the off-by-one behavior where the loop is either inclusive or exclusive of the bounds. For instance, in Python the `range` method excludes the upper limit — if you're not aware of this, it can be a source of bugs.

3. For large data types, pass-by-reference is preferred since the alternative is pass-by-value where you create a copy of the entire object. This copying of large objects can use a lot of memory and can be slow. Pass-by-reference passes a reference like a pointer, which lets the function access the original object. This is much faster and efficient and does not use memory. The most well-known drawback is that it allows the function to modify the original argument, so the values in the object that is being passed can be changed, which can introduce bugs if the programmer is wanting to maintain the values in the object. The way to mitigate this is by using the `const` reference to prevent modification — the `const` keyword makes the values read-only so values cannot be updated.

4. Variadic arguments allow a function to accept a variable number of positional or keyword arguments respectively. This is done through the use of `*args` for positional arguments and `**kwargs` for keyword arguments.

`*args` allows a function to accept any number of positional arguments. These arguments are collected into a tuple within the function:

```
```python
def add_all(*args):
 total = sum(args)
 return total
```

```
print(add_all(1, 2, 3, 4)) # Output: 10
```

```
...
```

`\*\*kwargs` allows a function to accept any number of keyword arguments. These arguments are collected into a dictionary within the function:

```
```python
def greet(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} => {value}")

greet(name="Elmer", age=22, country="USA")
```
```

**\*\*Output:\*\***

```
...
```

```
name => Elmer
age => 22
country => USA
...
```

5. Coroutines are generalized functions that can pause and resume their execution, maintaining their internal state between each pause. Unlike normal functions that start and end in one go, coroutines can yield control back and be resumed later. Coroutines are referred to as quasi-concurrent because the routines appear to run simultaneously, but in reality, only one coroutine executes at a time. The way to implement these in Python is by using the `yield` keyword, which will pause the function and return a value to the caller without destroying the function's local state. Later, the function can be resumed right after the `yield` statement. Alternatively, `async` and `await` are used in Python to write asynchronous, non-blocking code. `async def` defines a coroutine, and `await` pauses its execution until the awaited task (like a delay or I/O operation) is complete, without blocking the rest of the program. This allows multiple tasks to run concurrently in a clean and readable way, making it ideal for operations like web requests or file access.

```

```

6. The four pillars of OOP are **\*\*abstraction\*\***, **\*\*encapsulation\*\***, **\*\*inheritance\*\***, and **\*\*polymorphism\*\***.

Abstraction provides a cohesive interface that exposes only the relevant functionalities while loosely coupling components, hiding the inner complexity. In my program written in Go, there is an interface for the parent class (struct in Go) of all other product classes (structs) in the program. An interface is merely a collection of method signatures and a type "implements" an interface if it has methods that match the interface's method signatures. The interface abstracts

the specifics of how the all the products fulfill these methods — we just know that each product has these methods.

```
```go
type Product interface {
    GetProdID() int
    GetProdName() string
    GetPrice() float64
    GetReviewRate() float32
    SetProdID(int)
    SetProdName(string)
    SetPrice(float64)
    SetReviewRate(float32)

    GetProdTypeStr() string
    DisplayProdInfo()
    DisplayContentsInfo()
}
```
```

Encapsulation is information hiding and making it tamper-proof. Encapsulation in Go is done by controlling access to fields using capitalization. Fields or methods that start with a lowercase letter are private (unexported) and hidden from other packages, while those starting with an uppercase letter are public (exported). In my program, I used encapsulation in the `AudioProduct` struct by keeping `prodID` unexported to prevent it from being accessed or modified directly from outside the package, while exposing `Title` and `Price` for external use.

```
```go
type AudioProduct struct {
    prodID int    // unexported - hidden outside the package
    Title  string  // exported - accessible outside
    Price  float64 // exported - accessible outside
}
```
```

Inheritance enables the reusability of code and supports generalization/specialization, where general behavior is defined once and specialized in subtypes. Go doesn't use traditional inheritance but supports it via composition, which is the embedding of structs into other structs. This allows the outer struct to reuse the fields and methods of the embedded struct. While this is not traditional inheritance, it can often be more beneficial than the traditional way of inheriting.

Consider inheritance (like in Java or C++): types become tightly bound in a parent-child hierarchy. If your needs change, or if you want to reuse behavior in a different context, that rigid hierarchy can get in your way. With composition in Go, you can mix and match behaviors easily

by embedding different structs. Your new type isn't forever locked into a single "family tree." In my program, I created a base `Product` struct and then simply embedded that base struct in the `AudioProduct` struct, and the `AudioProduct` struct essentially inherits everything from the base struct.

```
```go
type AudioProduct struct {
    BaseProduct
    Genre string
}
...`
```

Polymorphism allows extensibility and customization through a single interface that supports different underlying types. In my program, I used polymorphism by defining a `Product` interface and implementing it across different types like `AudioProduct`, `VideoProduct`, and `Book`. Each type has its own customized version of the `DisplayProdInfo()` method, but they can all be treated as a `Product` in the cart.

```
```go
// Interface in product.go
type Product interface {
 GetProdID() int
 GetPrice() float64
 DisplayProdInfo()
}

// Implementation in audio.go
func (a *AudioProduct) DisplayProdInfo() {
 fmt.Printf("Audio Title: %s | Price: $%.2f", a.Title, a.Price)
}
...`
```